

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 8

Fitur Khusus Platform

Memisahkan kode platform dengan expect, actual, dan interface, meminta izin dengan sopan, dan memasang kamera, lokasi, atau notifikasi

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 8. Tahun 2026

Komponen	Keterangan
Pertemuan	8
Capaian	CPMK0501, menerapkan konsep pemrograman dalam bentuk fitur perangkat yang berjalan di lebih dari satu platform
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 8

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: foto bukti yang belum bisa diambil
3	Langkah 1: Kenapa kamera tidak bisa ditulis di commonMain
4	Langkah 2: Peningat expect dan actual
5	Langkah 3: Untuk fitur yang lebih besar, pakai interface
6	Langkah 4: Satu implementasi per platform, diberikan dari titik masuk
7	Langkah 5: Di mana izin dideklarasikan
8	Langkah 6: Empat langkah meminta izin
9	Langkah 7: Meminta izin kamera, dan saat izin ditolak
10	Langkah 8: Kamera, foto bukti laporan
11	Langkah 9: Lokasi, gedung terdekat
12	Langkah 10: Notifikasi, lalu pola yang sama untuk ketiganya
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 8. Sejauh ini hampir semua kode LaporanKampus ada di commonMain. Tetapi ada fitur yang mustahil ditulis sekali untuk semua platform: kamera, lokasi, dan notifikasi. Setiap sistem operasi punya caranya sendiri, dan setiap fitur itu butuh izin dari pengguna.

Modul ini terdiri dari tiga bagian. Langkah 1 sampai 4 membahas cara memisahkan kode platform dengan **expect dan actual** atau dengan **interface**. Langkah 5 sampai 7 membahas **izin**: di mana

dideklarasikan, kapan diminta, dan apa yang dilakukan saat ditolak. Langkah 8 sampai 10 memasang tiga fitur platform, yaitu kamera, lokasi, dan notifikasi, dengan pola yang sama.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Memisahkan kode platform dengan expect dan actual atau interface, lalu memberikan implementasinya lewat constructor	Langkah 1 sampai 4
2	Mendeklarasikan izin, memintanya saat dibutuhkan, dan tetap berfungsi saat ditolak	Langkah 5 sampai 7
3	Memasang kamera, lokasi, atau notifikasi di dua platform	Langkah 8 sampai 10

Capaian ini mendukung **CPMK0501**, dalam bentuk fitur perangkat yang berjalan di lebih dari satu platform. Kemampuan kedua dan ketiga juga yang diamati di observasi praktik minggu ini.

Yang perlu disiapkan

Proyek LaporKampus dari Modul 7, dan kalau ada, HP Android dengan kabel data. Kamera dan dialog izin terasa jauh lebih nyata di HP asli daripada di emulator. Untuk iOS dibutuhkan Mac dan Xcode. Kelompok tanpa Mac cukup memakai Android dan desktop.

PENTING Mengingat modul sebelumnya

expect dan actual sudah kita kenal di Modul 1 dan kita pakai untuk DriverFactory di Modul 7. Interface yang diberikan lewat constructor sudah kita pakai untuk Sumber<T> di Modul 2 dan 6. Modul ini memakai kedua pola itu untuk API perangkat.

PEMBUKA

Masalah pembuka: foto bukti yang belum bisa diambil

Petugas fasilitas mengeluh. Laporan seperti kran bocor di GKU 1 R201 sulit dicek tanpa foto. Tombol Ambil foto bukti sudah ada di form, tetapi belum melakukan apa-apa.

TEBAK DULU **Coba pikirkan**

Bagaimana Laporkampus membuka kamera di Android dan iOS? Dan kalau pengguna menolak izin kamera, apakah laporan tetap boleh dikirim?

JAWABAN **Arah jawabannya**

Dua ide yang sering muncul: mencari pustaka kamera di commonMain, atau menulis kodenya langsung di MainActivity. Ide pertama tidak berjalan, karena kamera tidak tersedia dengan cara yang sama di semua platform. Ide kedua membuat iOS dan desktop tidak punya fitur itu sama sekali. Jawaban yang kita pegang di modul ini: kebutuhannya ditulis di commonMain, caranya ditulis per platform. Untuk pertanyaan kedua, jawabannya **boleh**. Laporan tetap terkirim tanpa foto.

LANGKAH 1

Kenapa kamera tidak bisa ditulis di commonMain

Kebutuhan	Android	iOS	Desktop JVM
Ambil foto	TakePicturePreview	UIImagePickerController	Tidak ada kamera bawaan, pilih berkas
Lokasi	FusedLocationProviderClient	CLLocationManager	Tidak ada, pengguna memilih manual
Notifikasi	NotificationManagerCompat	UNUserNotificationCenter	SystemTray, atau cukup log
Izin	Manifest dan permintaan saat runtime	Info.plist dan dialog sistem	Umumnya tidak ada

commonMain hanya boleh memakai sesuatu yang tersedia di semua platform. Kelas di tabel ini hanya ada di SDK masing-masing, jadi kode bersama tidak bisa mengimpornya. Coba sendiri: ketik `import android.content.Context` di sebuah berkas di commonMain. Barisnya langsung merah.

ANALOGI Colokan listrik antarnegara

Charger laptop kalian sama, tetapi bentuk colokan di dinding berbeda di Indonesia, Inggris, dan Amerika. Kalian tidak membuat laptop baru untuk tiap negara. Kalian cukup membawa adapter yang berbeda. Kode di commonMain adalah laptopnya, dan kode di androidMain, iosMain, serta desktopMain adalah adaptornya.

LANGKAH 2

Pengingat expect dan actual

Untuk fungsi kecil, pasangan expect dan actual adalah cara paling ringkas. Ini contoh dari hands-on P9 di repo: membuat HP bergetar.

commonMain/Platform.kt

```
expect fun vibrate()
```

androidMain/Platform.android.kt

```
actual fun vibrate() {
    val ctx = AndroidAppContext.context ?: return
    // Ambil Vibrator atau VibratorManager dari ctx, lalu getar 200 ms.
    // Isi lengkapnya ada di Platform.android.kt pada hands-on P9.
}
```

desktopMain/Platform.desktop.kt

```
actual fun vibrate() {
    println("Desktop tidak mendukung getar")
}
```

Aturan	Artinya
Satu expect	Hanya deklarasi tanpa isi, di commonMain
Satu actual per target	Android, iOS, dan desktop masing-masing punya satu
Sama persis	Nama, paket, parameter, dan tipe kembalian harus sama

`AndroidAppContext` adalah object global yang diisi oleh MainActivity. README hands-on menyebutnya penyederhanaan. Di aplikasi sungguhan, Context sebaiknya diberikan lewat constructor, dan itu yang kita lakukan di Langkah 3 dan 4. Getar juga butuh izin VIBRATE di manifest, tetapi izin ini tidak perlu diminta saat runtime.

TEBAK DULU

Kita menambahkan `expect fun lokasiSekarang()`, tetapi actual-nya hanya ditulis di `androidMain`. Apa yang terjadi saat aplikasi Android dijalankan? Dan saat versi desktop dijalankan?

JAWABAN

Aplikasi Android **tetap berjalan**, karena kompilasi Android hanya membutuhkan `commonMain` dan `androidMain`. Versi desktop **gagal di-build** dengan pesan bahwa `expect lokasiSekarang` tidak punya actual declaration untuk target JVM. Android Studio biasanya sudah memberi garis merah di baris `expect` sebelum aplikasi dijalankan.

LANGKAH 3

Untuk fitur yang lebih besar, pakai interface

`expect` dan `actual` cocok untuk fungsi kecil. Untuk fitur yang butuh Context, punya keadaan sendiri, atau perlu dipalsukan saat pengujian, interface biasa lebih tepat. Dokumentasi Kotlin Multiplatform sendiri menyarankan memakai konstruksi bahasa biasa seperti interface bila sudah cukup.

Contoh kita adalah notifikasi saat status laporan berubah, karena kodenya pendek di ketiga platform.

```
commonMain/platform/Pemberitahu.kt
```

```
interface Pemberitahu {
    fun tampilkan(judul: String, isi: String)
}
```

```
commonMain/ui/DetailViewModel.kt
```

```
class DetailViewModel(
    private val id: Int,
    sumber: Sumber<Laporan>,
    private val pemberitahu: Pemberitahu
) : ViewModel() {
    fun kabari(status: String) =
        pemberitahu.tampilkan("Laporan #$id", status)
}
```

Kenapa interface	Akibatnya
Kelas biasa Kotlin	Boleh ada lebih dari satu implementasi per platform
Diberikan lewat constructor	ViewModel tidak tahu sedang berjalan di platform mana
Mudah dipalsukan	Versi palsu untuk unit test di Modul 10

Pola ini sama dengan Sumber di Modul 2 dan 6. SumberLaporanPalsu dan SumberLaporanJaringan sama-sama Sumber, dan layar tidak peduli yang mana. Pemberitahu pun begitu: ViewModel memanggil `tampilkan`, tidak peduli Android, iOS, atau desktop.

ANALOGI Sopir rental

Interface seperti memesan mobil rental beserta sopirnya. Kalian cukup bilang mau ke mana. Sopirnya bisa siapa saja, asal bisa menyetir. Di kota berbeda, sopirnya berbeda. Saat latihan, sopirnya bahkan bisa diganti boneka yang hanya mencatat rute, tanpa benar-benar menyetir. Itulah versi palsu untuk pengujian.

LANGKAH 4

Satu implementasi per platform, diberikan dari titik masuk

Implementasi tiap platform

androidMain/platform/PemberitahuAndroid.kt

```
class PemberitahuAndroid(private val context: Context) : Pemberitahu {
    override fun tampilkan(judul: String, isi: String) {
        val nm = NotificationManagerCompat.from(context)
        if (!nm.areNotificationsEnabled()) return
        val notif = NotificationCompat.Builder(context, "status")
            .setSmallIcon(R.drawable.ic_lapor)
            .setContentTitle(judul)
            .setContentText(isi)
            .build()
        nm.notify(judul.hashCode(), notif)
    }
}
```

desktopMain/platform/PemberitahuDesktop.kt

```
class PemberitahuDesktop : Pemberitahu {
    override fun tampilkan(judul: String, isi: String) =
        println("[Notifikasi] $judul: $isi")
}
```

Context Android sekarang masuk lewat constructor, bukan object global.

`areNotificationsEnabled()` bernilai false kalau izin notifikasi belum diberikan, sehingga fungsi berhenti tanpa crash. Android Studio mungkin tetap memberi peringatan MissingPermission, dan boleh dibiarkan karena izinnya sudah dicek. Implementasi desktop sengaja sederhana: cukup mencetak ke konsol supaya alur aplikasi tetap bisa diuji. Pemberitahu di `iosMain` memakai `UNUserNotificationCenter` dengan pola yang sama.

Diberikan dari titik masuk tiap platform

Setiap platform punya satu titik masuk, tempat aplikasi pertama kali dijalankan. Di situlah implementasi yang tepat dibuat dan diberikan ke App.

`androidMain/MainActivity.kt`

```
setContent {
    App(pemberitahu = PemberitahuAndroid(applicationContext))
}
```

`iosMain/MainViewController.kt`

```
fun MainViewController() = ComposeUIViewController {
    App(pemberitahu = PemberitahuIos())
}
```

`desktopMain/main.kt`

```
fun main() = application {
    Window(onCloseRequest = ::exitApplication) {
        App(pemberitahu = PemberitahuDesktop())
    }
}
```

App lalu meneruskannya ke ViewModel, misalnya `viewModel { DetailViewModel(id, sumber, pemberitahu) }`. Hanya tiga berkas ini yang tahu platform. Semua layar dan ViewModel tetap di `commonMain`.

	expect dan actual	Interface
Cocok untuk	Fungsi kecil seperti getar	Fitur dengan Context dan keadaan
Implementasi	Tepat satu per target	Bebas, termasuk versi palsu
Saat pengujian	Sulit diganti	Cukup kirim versi palsu

Meneruskan objek dari App ke setiap ViewModel memang terasa merepotkan kalau dependensinya banyak. Di Modul 10, pustaka Koin mengambil alih pekerjaan ini, dan latihan 3 minggu ini memberi gambaran awalnya.

LANGKAH 5

Di mana izin dideklarasikan

Fitur seperti kamera dan lokasi menyentuh privasi pengguna, jadi harus dideklarasikan lebih dulu.

androidMain/AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA" />
```

iosApp/iosApp/Info.plist

```
<key>NSCameraUsageDescription</key>
<string>Foto membantu petugas melihat kerusakan fasilitas.</string>
```

Fitur	Android, di AndroidManifest.xml	iOS, di Info.plist
Kamera	CAMERA	NSCameraUsageDescription
Lokasi	ACCESS_FINE_LOCATION dan ACCESS_COARSE_LOCATION	NSLocationWhenInUseUsageDescription
Notifikasi	POST_NOTIFICATIONS, untuk Android 13 ke atas	Tanpa key, diminta lewat kode

Deklarasi saja belum cukup untuk izin berbahaya seperti kamera dan lokasi. Di Android, izin itu tetap harus diminta saat aplikasi berjalan. Di iOS, teks di Info.plist tampil di dialog sistem, jadi tulis dalam bahasa pengguna, bukan bahasa programmer.

PENTING Key yang hilang di iOS

Kalau key NSCameraUsageDescription tidak ada, aplikasi iOS langsung ditutup sistem saat mencoba mengakses kamera, tanpa dialog apa pun. Folder iosApp tidak disertakan di hands-on P9, tetapi ada di proyek baru dari kmp.jetbrains.com.

LANGKAH 6

Empat langkah meminta izin

Langkah	Android	iOS
1 Cek status	ContextCompat.checkSelfPermission	AVCaptureDevice.authorizationStatusForMediaType

Langkah	Android	iOS
2 Perlu penjelasan ?	shouldShowRequestPermissionRationale	Teks dari Info.plist tampil di dialog
3 Minta	Launcher RequestPermission	AVCaptureDevice.requestAccessForMediaType
4 Kalau ditolak	Boleh diminta lagi. Setelah dua kali ditolak, dialog tidak muncul lagi	Dialog hanya muncul sekali. Selanjutnya hanya lewat Pengaturan

Aturan dua kali ditolak berlaku sejak Android 11. Di iOS, dialog hanya muncul sekali seumur pemasangan aplikasi. Karena itu, mintalah izin **tepat saat pengguna menekan tombol yang membutuhkannya**, bukan saat aplikasi baru dibuka. Dengan begitu pengguna paham kenapa izin itu diminta.

ANALOGI **Meminjam motor teman**

Kalian meminta izin meminjam motor teman tepat saat butuh, sambil menjelaskan mau ke mana. Kalian tidak meminta izin itu di hari pertama berkenalan. Kalau teman menolak dua kali, berhentilah bertanya. Kalian naik angkot, dan teman itu bisa menawarkan sendiri kapan pun ia mau.

TEBAK DULU

Pelapor sudah dua kali menekan Jangan izinkan. Lalu ia menekan Ambil foto bukti sekali lagi. Apa yang muncul di layar? Apa yang seharusnya dilakukan Laporkampus?

JAWABAN

Tidak muncul apa-apa dari sistem. Launcher langsung mengembalikan false tanpa dialog. Kalau aplikasi tidak menangani false, pengguna mengira tombolnya rusak. Yang seharusnya dilakukan: form tetap bisa dikirim tanpa foto, ada pesan singkat bahwa izin kamera dimatikan, dan ada tombol Buka Pengaturan bagi yang memang ingin mengaktifkannya.

LANGKAH 7

Meminta izin kamera, dan saat izin ditolak

Kode di Compose Android

```
androidMain/ui/TombolFoto.kt
```

```

@Composable
fun TombolFoto(onFoto: (Bitmap?) → Unit) {
    var ditolak by remember { mutableStateOf(false) }
    val kamera = rememberLauncherForActivityResult(
        ActivityResultContracts.TakePicturePreview()
    ) { onFoto(it) }
    val izin = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { boleh →
        ditolak = !boleh
        if (boleh) kamera.launch(null)
    }
    Button(onClick = { izin.launch(Manifest.permission.CAMERA) }) {
        Text("Ambil foto bukti")
    }
    if (ditolak) Text("Tanpa foto, laporan tetap terkirim.")
}

```

Bagian	Gunanya
<code>rememberLauncherForActivityResult</code>	Menyiapkan peluncur yang hasilnya kembali ke composable ini
<code>RequestPermission</code>	Menampilkan dialog bila perlu, hasilnya Boolean
<code>TakePicturePreview</code>	Membuka aplikasi kamera, hasilnya Bitmap kecil
<code>ditolak</code>	State untuk pesan, sementara form tetap berjalan

Kalau izin sudah diberikan sebelumnya, `RequestPermission` langsung mengembalikan `true` tanpa dialog. Izin diminta lebih dulu karena kalau `CAMERA` dideklarasikan di manifest, membuka kamera tanpa izin akan gagal dengan `SecurityException`. `TakePicturePreview` hanya memberi gambar kecil, cukup untuk foto bukti. Untuk foto ukuran penuh, dipakai `TakePicture` dengan Uri dari `FileProvider`.

Tiga prinsip saat izin ditolak

Prinsip	Artinya di Laporan Kampus
Tetap berfungsi	Laporan tetap bisa dikirim tanpa foto. Jangan mengunci layar hanya karena satu izin
Jelaskan dengan spesifik	Tulis gunanya, misalnya foto membantu petugas melihat kerusakan, bukan sekadar izin diperlukan
Pengaturan bila diminta	Setelah ditolak permanen, beri tombol Buka Pengaturan di tempat fitur itu, tanpa memaksa

Panduan ini sejalan dengan dokumentasi izin Android Developers: kurangi fitur yang hilang, jangan memblokir tampilan, dan hormati pilihan pengguna. Yang perlu dihindari: dialog penjelasan yang muncul setiap kali aplikasi dibuka, atau layar penuh yang tidak bisa ditutup.

```
androidMain/ui/BukaPengaturan.kt
```

```
fun bukaPengaturan(context: Context) {
    val intent = Intent(
        Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
        Uri.fromParts("package", context.packageName, null)
    )
    context.startActivity(intent)
}
```

Di composable, ambil Context dengan `LocalContext.current`, lalu tampilkan TextButton Buka Pengaturan di samping pesan penolakan. Di iOS, halaman pengaturan aplikasi dibuka dengan `UIApplicationOpenSettingsURLString`.

ANALOGI Lift yang sedang rusak

Gedung yang baik tetap punya tangga saat lift rusak. Tidak ada yang menutup gedung hanya karena satu lift berhenti. Begitu pula aplikasi: satu izin ditolak, fiturnya berkurang, tetapi pengguna tetap bisa menyelesaikan urusannya.

LANGKAH 8

Kamera, foto bukti laporan

Kita ingin tombol di `FormLaporanScreen`, yang ada di `commonMain`, bisa membuka kamera. Masalahnya, peluncur kamera Android harus dibuat di dalam composable dengan `rememberLauncherForActivityResult`. Karena itu, expect di sini berbentuk **fungsi composable yang mengembalikan fungsi** untuk dipanggil tombol.

`commonMain/platform/AmbilFoto.kt`

```
@Composable
expect fun rememberAmbilFoto(hasil: (ByteArray?) → Unit): () → Unit
```

`commonMain/ui/FormLaporanScreen.kt`

```
val ambilFoto = rememberAmbilFoto { vm.lampirkanFoto(it) }
Button(onClick = ambilFoto) { Text("Ambil foto bukti") }
state.foto?.let { Text("Foto terlampir, ${it.size / 1024} KB") }
```

Hasilnya bertipe **ByteArray**, yaitu deretan byte mentah, karena tipe ini tersedia di semua platform. Bitmap hanya ada di Android, sedangkan UIImage hanya ada di iOS.

ANALOGI Amplop standar

ByteArray seperti amplop ukuran standar yang diterima semua kantor pos. Isi fotonya boleh diambil dengan kamera merek apa pun, asal dimasukkan ke amplop yang sama, sehingga bisa dikirim ke mana saja tanpa diubah lagi.

Actual di Android

androidMain/platform/AmbilFoto.android.kt

```
@Composable
actual fun rememberAmbilFoto(hasil: (ByteArray?) → Unit): () → Unit {
    val kamera = rememberLauncherForActivityResult(
        ActivityResultContracts.TakePicturePreview()
    ) { bitmap →
        hasil(bitmap?.let { b →
            val keluar = ByteArrayOutputStream()
            b.compress(Bitmap.CompressFormat.JPEG, 85, keluar)
            keluar.toByteArray()
        })
    }
    val izin = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { boleh → if (boleh) kamera.launch(null) else hasil(null) }
    return { izin.launch(Manifest.permission.CAMERA) }
}
```

Bitmap dikompres ke format JPEG dengan kualitas 85, lalu diubah menjadi ByteArray. Kalau izin ditolak atau pengguna membatalkan kamera, `hasil(null)` dipanggil, dan form tetap berjalan tanpa foto.

Actual di desktop

desktopMain/platform/AmbilFoto.desktop.kt

```
@Composable
actual fun rememberAmbilFoto(hasil: (ByteArray?) → Unit): () → Unit = {
    val dialog = FileDialog(null as Frame?, "Pilih foto bukti", FileDialog.LOAD)
    dialog.isVisible = true
    val berkas = dialog.file?.let { File(dialog.directory, it) }
    hasil(berkas?.readBytes())
}
```

Desktop tidak punya kamera bawaan, jadi kita membuka jendela pemilih berkas. `dialog.isVisible = true` menunggu sampai pengguna memilih atau membatalkan. Kalau dibatalkan, `dialog.file` bernilai null.

Actual di iOS

Untuk kelompok yang punya Mac, actual di iosMain memakai UIImagePickerControllerController untuk membuka kamera, didahului pemeriksaan izin dengan AVCaptureDevice seperti tabel di Langkah 6.

Gambar yang didapat diubah menjadi JPEG, lalu menjadi ByteArray. Kelompok tanpa Mac cukup menulis actual untuk Android dan desktop.

ViewModel menyimpan foto

commonMain/ui/FormLaporanViewModel.kt

```
// di FormLaporanUiState tambahkan: val foto: ByteArray? = null
fun lampirkanFoto(data: ByteArray?) {
    if (data != null) _uiState.update { it.copy(foto = data) }
}
```

Foto disimpan di UiState form, lalu nanti dikirim bersama laporan. Sesuai Langkah 1 Modul 7, foto yang perlu disimpan di perangkat ditaruh sebagai berkas, dan database hanya mencatat alamatnya.

LANGKAH 9

Lokasi, gedung terdekat

Untuk tema yang memakai lokasi, kebutuhannya kita tulis sebagai interface, karena mengambil lokasi butuh waktu dan bisa gagal.

commonMain/platform/PenyediaLokasi.kt

```
data class Koordinat(val lat: Double, val lon: Double)

interface PenyediaLokasi {
    suspend fun sekarang(): Koordinat? // null bila ditolak atau belum didapat
}
```

commonMain/domain/GedungTerdekat.kt

```
import kotlin.math.pow

fun gedungTerdekat(posisi: Koordinat, gedung: Map<String, Koordinat>): String? =
    gedung.minByOrNull { (_, g) →
        (g.lat - posisi.lat).pow(2) + (g.lon - posisi.lon).pow(2)
    }?.key
```

`minByOrNull` mencari gedung dengan nilai terkecil, lalu `?.key` mengambil namanya. Untuk jarak sependek area kampus, selisih kuadrat lintang dan bujur sudah cukup untuk membandingkan mana yang paling dekat. Hasilnya bukan jarak dalam meter. Kalau butuh meter, pakai rumus haversine.

TEBAK DULU

Dengan angka contoh berikut, bukan koordinat sebenarnya, gedung apa yang dikembalikan?

Main.kt

```
val gedung = mapOf(
    "GKU 1" to Koordinat(1.0, 1.0),
    "GKU 2" to Koordinat(1.0, 3.0),
    "Labtek 1" to Koordinat(4.0, 1.0)
)
println(gedungTerdekat(Koordinat(1.2, 2.4), gedung))
```

JAWABAN

GKU 2. Nilainya untuk GKU 1 adalah $0,04 + 1,96 = 2,0$. Untuk GKU 2 adalah $0,04 + 0,36 = 0,4$. Untuk Labtek 1 adalah $7,84 + 1,96 = 9,8$. Nilai terkecil milik GKU 2. Karena perhitungan ini ada di commonMain, ia bisa diuji tanpa GPS sama sekali.

Implementasi	Memakai
Android	FusedLocationProviderClient, atau LocationManager bawaan Android
iOS	CLLocationManager dengan izin saat aplikasi dipakai
Desktop	Kembalikan null, pengguna memilih gedung dari daftar
Versi palsu untuk pengujian	Kembalikan koordinat tetap

Koordinat gedung diisi kelompok sendiri dari peta, lalu disimpan di database atau sebagai konstanta. Nilai null dipakai untuk dua keadaan, yaitu izin ditolak dan lokasi belum didapat. Di keduanya, pengguna memilih gedung dari daftar.

LANGKAH 10

Notifikasi, lalu pola yang sama untuk ketiganya

Notifikasi status laporan

Di Android, notifikasi harus masuk ke sebuah **channel**. Channel dibuat sekali saat aplikasi mulai, misalnya di MainActivity atau kelas Application.

androidMain/LaporKampusApp.kt

```
val kanal = NotificationChannelCompat.Builder(
    "status", NotificationManagerCompat.IMPORTANCE_DEFAULT
).setName("Status Laporan").build()
NotificationManagerCompat.from(context).createNotificationChannel(kanal)
```

Langkah	Android	iOS
Izin	POST_NOTIFICATIONS diminta saat runtime, Android 13 ke atas	requestAuthorizationWithOptions di UNUserNotificationCenter
Tampilkan	NotificationCompat.Builder lalu notify	UNNotificationRequest lalu addNotificationRequest

Yang kita buat adalah **notifikasi lokal**: aplikasi sendiri yang memunculkannya saat melihat status berubah, misalnya setelah menyegarkan data dari server di Modul 7. Notifikasi yang dikirim server saat aplikasi tertutup, lewat Firebase Cloud Messaging atau APNs, di luar cakupan mata kuliah ini.

ANALOGI Alarm dan SMS bank

Notifikasi lokal seperti alarm yang kalian pasang sendiri di HP: HP yang memutuskan kapan berbunyi. Notifikasi dari server seperti SMS dari bank: pihak luar yang mengirim, walau kalian tidak sedang membuka aplikasinya.

Minta izin notifikasi setelah pengguna mengirim laporan pertamanya, saat alasannya jelas, bukan saat aplikasi pertama kali dibuka. Kalau ditolak, status tetap bisa dilihat di layar detail.

Satu fitur platform per tema

Fitur	Contoh di Laporkampus	Tema di Bank Proyek
Kamera	Foto bukti kerusakan di form laporan	7 tema, misalnya Barang hilang dan temuan kampus
Lokasi	Gedung terdekat terisi otomatis	6 tema, misalnya Tebengan ke kampus
Notifikasi	Kabar saat status berubah menjadi Diproses	7 tema, misalnya Pengingat tugas kuliah

Pola yang sama untuk ketiganya

	Kamera	Lokasi	Notifikasi
Diminta saat	Ketuk Ambil foto bukti	Ketuk Gedung terdekat	Laporan pertama terkirim
Kalau ditolak	Kirim tanpa foto	Pilih gedung dari daftar	Status dilihat di layar detail
Di desktop	Pilih berkas foto	Pilih gedung dari daftar	Cetak ke log

	Kamera	Lokasi	Notifikasi
Versi palsu untuk pengujian	Foto contoh tetap	Koordinat tetap	Simpan pesan ke list

Ketiga fitur setara bobotnya. Yang dinilai bukan fiturnya, tetapi pemisahan dengan expect dan actual atau interface, izin yang ditangani, dan fitur yang berjalan di dua platform. Baris terakhir tabel sudah menyiapkan Modul 10: karena fitur platform ada di balik interface atau expect, unit test bisa memakai versi palsu tanpa kamera atau GPS.

PRAKTIK

Latihan mandiri

Ada tiga latihan di folder P9 Platform Specific Features Hands-on. Menu di App.kt berpindah antara latihan dan solusi. Latihan 1 dan 2 yang diamati.

Latihan	Berkas	Yang dilatih
1	Handson1Vibrate.kt	Panggil <code>vibrate()</code> dari tombol, lalu bandingkan hasilnya di Android dan desktop
2	Handson2Permission.kt	Tampilkan status izin kamera dengan pesan berbeda untuk diberikan dan ditolak
3	Handson3Di.kt	Ambil <code>GreetingRepository</code> dengan <code>koinInject</code> , sebagai pemanasan Modul 10

CONTOH Petunjuk latihan 1

`expect` dan `actual`-nya sudah tersedia di `Platform.kt`, seperti Langkah 2. Tugas kalian memanggilnya dari tombol. Uji di Android: getar terasa di HP asli. Beberapa emulator tidak mensimulasikan getar, jadi anggap log sebagai bukti fungsinya terpanggil. Di desktop, yang muncul adalah baris di konsol.

CONTOH Petunjuk latihan 2

Ikuti empat langkah di Langkah 6. Tampilkan pesan yang berbeda untuk izin diberikan dan ditolak, lalu uji keduanya. Untuk menguji ulang dialog izin yang sudah ditolak permanen, hapus data aplikasi atau `uninstall` lalu pasang lagi. Di desktop, izin selalu dianggap diberikan.

CONTOH Petunjuk latihan 3

Panggil `initKoin` sebelum `App()` di titik masuk, lalu ambil `GreetingRepository` dengan `koinInject()` di dalam `composable`. Tidak apa-apa kalau belum paham semuanya. Koin dibahas lengkap di Modul 10. Yang sudah selesai ketiga latihan, buat `TombolFoto` dengan izin asli seperti Langkah 7 di `androidMain`.

Yang diamati

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	<code>expect</code> dan <code>actual</code> berjalan di Android dan iOS, atau Android dan desktop bagi yang tanpa Mac, dan izin yang ditolak pun ditangani

Level	Skor	Yang terlihat
Baik	70 sampai 84	Berjalan di dua platform, penolakan izin belum ditangani
Cukup	55 sampai 69	Hanya berjalan di satu platform
Kurang	di bawah 55	Belum ada fitur platform yang berjalan

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Expected function has no actual declaration	Satu target belum punya actual, atau tanda tangannya berbeda	Samakan nama, parameter, dan tipe kembalian
SecurityException saat membuka kamera	CAMERA ada di manifest, tetapi belum diizinkan	Minta izin dulu, baru buka kamera
Dialog izin tidak muncul	Izin belum ada di manifest, atau sudah dua kali ditolak	Periksa manifest, lalu izinkan lewat Pengaturan
Aplikasi iOS tertutup saat kamera dibuka	NSCameraUsageDescription belum ada	Tambahkan key itu di Info.plist
NoBeanDefFoundException	koinInject dipanggil sebelum initKoin	Panggil initKoin sebelum App()
Unresolved reference: android di commonMain	Kelas Android diimpor dari kode bersama	Pindahkan kode itu ke androidMain, sisakan expect atau interface di commonMain

PRAKTIK

Bawa ke proyekmu

Pilih satu fitur platform sesuai tema kelompok di Bank Proyek. Bagi kerja: satu anggota interface dan versi palsu, satu anggota implementasi Android, satu anggota implementasi kedua dan jalur saat izin ditolak. Mulai dengan mengisi tabel pola yang sama di Langkah 10 untuk fitur kalian, dan taruh tabelnya di README.

- Interface atau expect.** Tulis kebutuhan fitur di commonMain, lengkap dengan versi palsu.
- Dua implementasi.** Android dan iOS, atau Android dan desktop bagi kelompok tanpa Mac.
- Izin di dua tempat.** Deklarasi di manifest dan Info.plist, diminta saat tombolnya ditekan.
- Jalur saat ditolak.** Tentukan apa yang tetap bisa dilakukan pengguna tanpa fitur itu.

CONTOH Contoh dari Laporan Kampus

rememberAmbilFoto di commonMain, dengan actual di androidMain dan desktopMain. Tanpa izin kamera, laporan tetap terkirim tanpa foto, dengan pesan singkat dan tombol Buka Pengaturan.

Checkpoint minggu 8

No	Yang diselesaikan
1	Kamera, lokasi, atau notifikasi lewat expect dan actual atau interface
2	Berjalan di Android dan iOS, atau Android dan desktop
3	Diminta saat dibutuhkan, aplikasi tetap berjalan saat izin ditolak

Push ke repository kelompok. README memuat klip pendek saat izin diberikan dan saat ditolak. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Fitur ini menjadi bagian kriteria Fitur lanjutan terintegrasi di tes lisan 2.

PENUTUP

Rangkuman

API perangkat seperti kamera, lokasi, dan notifikasi hanya ada di SDK masing-masing platform, jadi tidak bisa ditulis langsung di `commonMain`. Kebutuhan ditulis di `commonMain`, caranya ditulis per platform.

expect dan actual cocok untuk fungsi kecil, dengan tepat satu actual per target yang tandanya sama persis. **Interface** cocok untuk fitur dengan Context dan keadaan. Implementasinya dibuat di titik masuk tiap platform dan diberikan lewat constructor, sehingga mudah diganti versi palsu saat pengujian.

Izin dideklarasikan di `AndroidManifest.xml` dan `Info.plist`, lalu diminta tepat saat tombolnya ditekan. Setelah dua kali ditolak di Android, atau sekali di iOS, dialog tidak muncul lagi. Aplikasi harus tetap berfungsi saat izin ditolak, menjelaskan gunanya dengan spesifik, dan menawarkan Pengaturan tanpa memaksa.

Kamera memakai `expect` berbentuk `composable` yang mengembalikan fungsi, dengan hasil `ByteArray`. Lokasi memakai `interface` dengan perhitungan gedung terdekat di `commonMain`. Notifikasi lokal butuh channel di Android dan izin di Android 13 ke atas serta iOS.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kenapa kode kamera tidak bisa ditulis di commonMain?
2. expect sebuah fungsi hanya punya actual di androidMain. Target mana yang gagal di-build?
3. Kapan memilih interface daripada expect dan actual?
4. Di mana izin kamera dideklarasikan untuk Android dan untuk iOS?
5. Kenapa izin sebaiknya diminta saat tombol ditekan, bukan saat aplikasi pertama dibuka?
6. Apa yang harus dilakukan aplikasi saat izin ditolak?
7. Kenapa hasil rememberAmbilFoto bertipe ByteArray, bukan Bitmap?

JAWABAN Kunci jawaban

1. API kamera berbeda di tiap platform dan hanya ada di SDK masing-masing, sehingga kode bersama tidak bisa mengimpornya.
2. Semua target selain Android, misalnya desktop dan iOS. Target Android tetap berhasil.
3. Saat fitur butuh Context atau keadaan, punya lebih dari satu implementasi, atau perlu dipalsukan saat pengujian.
4. Di AndroidManifest.xml dengan uses-permission CAMERA, dan di Info.plist dengan key NSCameraUsageDescription.
5. Supaya pengguna paham alasannya, dan karena dialog izin hanya muncul terbatas: sekali di iOS dan dua kali penolakan di Android.
6. Tetap berfungsi tanpa fitur itu, menjelaskan gunanya dengan spesifik, dan menawarkan tombol Buka Pengaturan tanpa memaksa.
7. Karena ByteArray tersedia di semua platform, sedangkan Bitmap hanya ada di Android.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
kotlinlang.org, Expected and actual declarations	Aturan expect dan actual, serta kapan interface lebih tepat
developer.android.com, Request runtime permissions	Alur izin Android dan aturan penolakan
developer.android.com, Create a notification	Channel dan NotificationCompat
developer.android.com, Request location permissions	Izin lokasi di Android
developer.apple.com, Requesting authorization for media capture	Izin kamera di iOS

CONTOH Masalah Pertemuan 9

Pelapor sering menulis judul seadanya, misalnya rusak. Bisakah AI membantu menebak kategori, gedung, dan tingkat urgensi dari deskripsi, tanpa mengambil alih keputusan pengguna? Tab Asisten yang kosong sejak Modul 5 akhirnya diisi. Siapkan akun Google untuk Google AI Studio, dan ingat sejak sekarang: kunci API tidak boleh masuk repository.